Homework 10

All forests use RLT 3.2.6 from CRAN. Its defaults differ from the RLT 6.1.1 used in lecture, so `fit_rf()` sets them to match 6.1.1: best splits (`split.gen = "best"`), no meaningful child-size limit (`alpha = 0.001`), and `nmin` = node size + 1, because 3.2.6 splits a node of size `nmin` while the homework stops at that size.

Question 1 Answers

Part a

The seed is 43210.

```
library(MASS)
library(RLT)

fit_rf <- function(x, y, model, ntrees, mtry, node_size, resample_prob = 1, ...) {
  capture.output(
    fit <- RLT(
      x, y, model = model, ntrees = ntrees, mtry = mtry, nmin = node_size + 1,
      alpha = 0.001, split.gen = "best", resample.prob = resample_prob,
      replacement = TRUE, reinforcement = FALSE, ...
    )
  )
  fit
}

set.seed(43210)
n_rep <- 100
Sigma <- matrix(0.3, 5, 5) + diag(0.7, 5)
gen_x <- function(n) {
  x <- mvrnorm(n, rep(0, 5), Sigma)
  colnames(x) <- paste0("X", 1:5)
  x
}

X_test <- gen_x(200)
f_test <- X_test[, 1] + X_test[, 3] + X_test[, 5]
train_sets <- lapply(1:n_rep, function(r) {
  X <- gen_x(200)
  list(X = X, y = X[, 1] + X[, 3] + X[, 5] + rnorm(200))
})

bias_variance <- function(mtry, node, prob = 1) {
  preds <- t(sapply(train_sets, function(d) {
    fit <- fit_rf(
      d$X, d$y, "regression", 500, mtry, node,
      resample_prob = prob, importance = FALSE
    )
    predict(fit, X_test)$Prediction
  }))
  c(
    bias2 = mean((colMeans(preds) - f_test)^2),
    variance = mean(apply(preds, 2, var))
  )
```

```
}

node_grid <- c(1, 5, 10, 20, 30)
mtry_grid <- 1:5
node_results <- t(sapply(node_grid, function(s) bias_variance(3, s)))
mtry_results <- t(sapply(mtry_grid, function(m) bias_variance(m, 10)))

plot_bv <- function(x, results, xlab) {
  plot(
    x, results[, "bias2"], type = "b", pch = 16, lwd = 2, col = "#1F5A94",
    ylim = c(0, 1.2 * max(results)), xlab = xlab, ylab = "Average over test points"
  )
  lines(x, results[, "variance"], type = "b", pch = 17, lwd = 2, col = "#C84A16")
  legend(
    "top", bty = "n", lwd = 2, pch = c(16, 17), horiz = TRUE,
    col = c("#1F5A94", "#C84A16"), legend = c("Squared bias", "Variance")
  )
}

print(round(cbind(node = node_grid, node_results, total = rowSums(node_results)), 3))
```

```
     node bias2 variance total
[1,]    1 0.164    0.206 0.369
[2,]    5 0.167    0.199 0.366
[3,]   10 0.179    0.186 0.365
[4,]   20 0.231    0.168 0.399
[5,]   30 0.307    0.164 0.472
```

```
print(round(cbind(mtry = mtry_grid, mtry_results, total = rowSums(mtry_results)), 3))
```

```
     mtry bias2 variance total
[1,]    1 0.493    0.116 0.609
[2,]    2 0.240    0.149 0.389
[3,]    3 0.180    0.185 0.365
[4,]    4 0.162    0.217 0.378
[5,]    5 0.154    0.239 0.393
```

```
plot_bv(node_grid, node_results, "Node size")
```

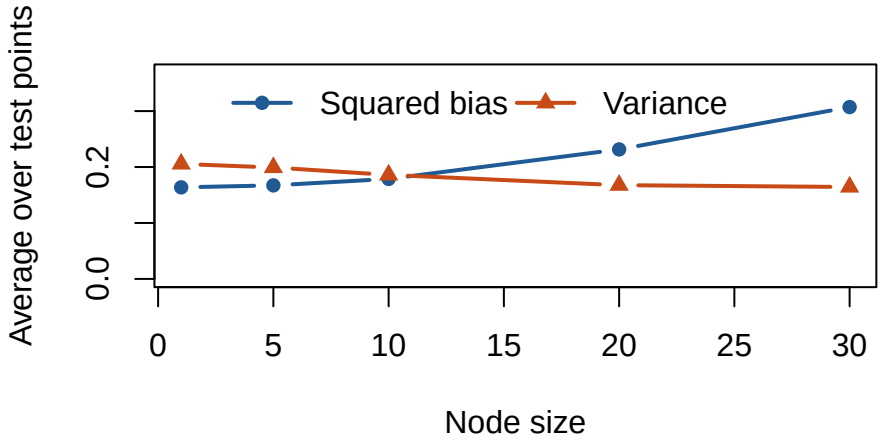


Figure 1: Average squared bias and variance by node size, with mtry fixed at 3.

```
plot_bv(mtry_grid, mtry_results, "mtry")
```

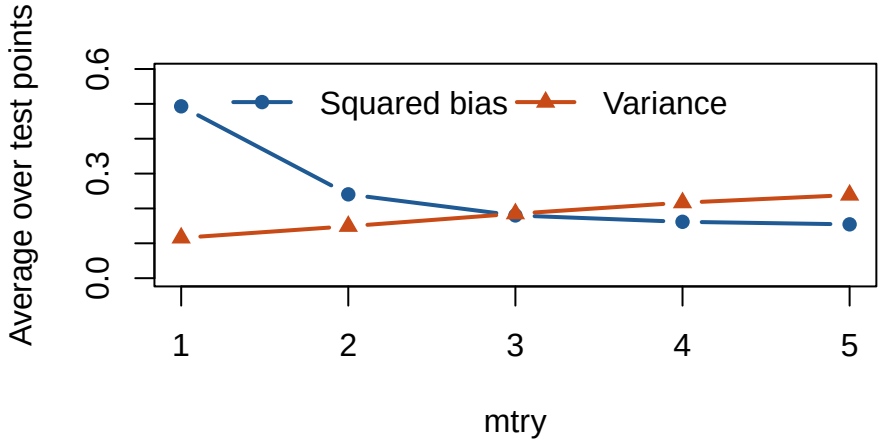


Figure 2: Average squared bias and variance by mtry, with node size fixed at 10.

Part b

```
sample_100 <- t(sapply(node_grid, function(s) bias_variance(3, s, prob = 0.5)))
print(round(cbind(node = node_grid, sample_100, total = rowSums(sample_100)), 3))
```

```
     node bias2 variance total
[1,]    1 0.229    0.140 0.369
[2,]    5 0.237    0.133 0.370
[3,]   10 0.270    0.127 0.397
```

```
[4,]   20 0.403    0.121 0.525
[5,]   30 0.589    0.122 0.711
```

- **Large node size:** Node size 30 gives a coarser, smoother fit. Compared with node size 5, variance falls only from 0.199 to 0.164, but squared bias nearly doubles from 0.167 to 0.307, so the total error is worse. The true function is linear, and large terminal nodes flatten that slope.
- **Large `mtry`:** The benefit is lower bias, since a strong predictor is nearly always a candidate (mtry 5 has squared bias 0.154, while mtry 1 has 0.493). The drawback is higher variance (0.239 vs. 0.116), because the trees look more alike and averaging removes less of their variation. Three of the five predictors matter, and total error is lowest at mtry 3.
- **Sampling 100 observations:** I would expect the preferred node size to shift smaller, and the simulation agrees. With half the data, a node of a given size covers twice the share of each tree's sample, so the same node size is coarser. Squared bias at node size 30 rises from 0.307 to 0.589. The smaller sample already lowers variance (0.140 vs. 0.206 at node size 1) by making the trees less correlated, so large nodes add little. Total error is lowest at node sizes 1 to 5 with 100 observations, compared with node sizes 5 to 10 with 200.

Question 2 Answers

Part a

```r
mnist <- read.csv(gzfile("data/mnist-first2000.csv.gz"))
pixels <- setdiff(names(mnist), "Digit")
train <- subset(mnist[1:1000, ], Digit %in% c(1, 6, 7))
test <- subset(mnist[1001:2000, ], Digit %in% c(1, 6, 7))

pca <- prcomp(train[, pixels], center = TRUE, scale. = FALSE)
Z_train <- pca$x[, 1:20]
Z_test <- predict(pca, test[, pixels])[, 1:20]
y_train <- factor(train$Digit)
y_test <- factor(test$Digit, levels = levels(y_train))

fit_mnist <- function(mtry, node, importance = FALSE) {
  set.seed(43220)
  fit_rf(
    Z_train, y_train, "classification", 1000, mtry, node,
    track.obs = TRUE, importance = importance
  )
}

# RLT 3.2.6 does not return forest OOB predictions, so average each tree's
# class probabilities over the observations it left out of its bootstrap sample.
oob_error <- function(fit) {
  tree_probs <- predict(fit, Z_train)$AllPrediction
  prob_sum <- matrix(0, nrow(Z_train), nlevels(y_train))
  for (t in seq_along(tree_probs)) {
    oob <- fit$ObsTrack[, t] == 0
    prob_sum[oob, ] <- prob_sum[oob, ] + tree_probs[[t]][oob, ]
  }
  mean(levels(y_train)[max.col(prob_sum, ties.method = "first")] != y_train)
}

default_fit <- fit_mnist(mtry = 10, node = 5)
print(c(n_train = nrow(train), n_test = nrow(test), oob_error = oob_error(default_fit)))
```

```
     n_train       n_test    oob_error
327.00000000 317.00000000   0.03975535
```

The defaults are the RLT 6.1.1 values, `mtry` $= \lfloor 20/2 \rfloor = 10$ and node size 5. The OOB misclassification rate is about 3.98%.

Part b

```r
grid <- expand.grid(
  node = c(1, 3, 5, 10, 20, 30),
  mtry = c(1, 2, 3, 4, 6, 10, 15, 20)
)
grid$oob <- mapply(
  function(m, s) oob_error(fit_mnist(m, s)),
  grid$mtry, grid$node
)
print(round(xtabs(oob ~ node + mtry, grid), 4))
```

```
    mtry
node      1      2      3      4      6     10     15     20
  1  0.0183 0.0306 0.0367 0.0367 0.0367 0.0398 0.0489 0.0489
  3  0.0275 0.0336 0.0336 0.0367 0.0367 0.0398 0.0489 0.0489
  5  0.0275 0.0306 0.0367 0.0367 0.0367 0.0398 0.0489 0.0489
  10 0.0245 0.0398 0.0367 0.0367 0.0398 0.0398 0.0520 0.0489
  20 0.0245 0.0398 0.0367 0.0398 0.0398 0.0428 0.0489 0.0489
  30 0.0336 0.0398 0.0398 0.0367 0.0367 0.0459 0.0489 0.0489
```

```r
# Break ties toward the larger node size, then the smaller mtry.
best <- grid[order(grid$oob, -grid$node, grid$mtry)[1], ]
print(best)
```

```
  node mtry        oob
1    1    1 0.01834862
```

```r
final_fit <- fit_mnist(best$mtry, best$node, importance = TRUE)
errors <- final_fit$AllError
importance <- colMeans(errors[, colnames(Z_train)] - errors[, "nonPermute"])

barplot(
  importance[order(importance, decreasing = TRUE)], las = 2, cex.names = 0.7,
  col = "#1F5A94", border = NA, ylab = "Increase in OOB error"
)
```

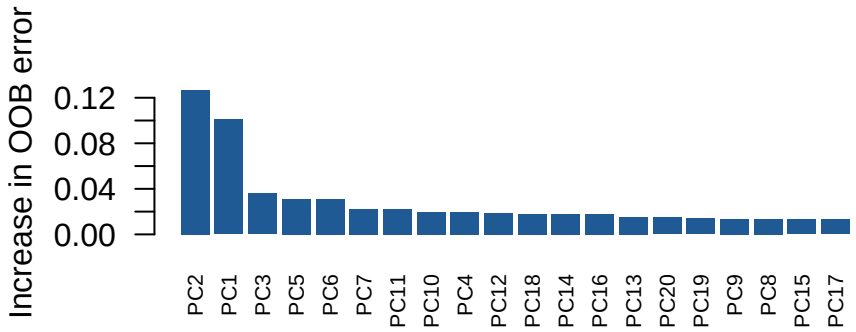


Figure 3: OOB permutation importance of the selected forest.

```
print(round(sort(importance, decreasing = TRUE)[1:6], 4))
```

```
   PC2    PC1    PC3    PC5    PC6    PC7
0.1269 0.1009 0.0358 0.0313 0.0313 0.0224
```

The selected setting is `mtry` 1 and node size 1, with OOB error 1.83% (6 of 327), compared with 3.98% for the default. Error mostly grows with `mtry`, up to about 5% at `mtry` 15 and 20: with large `mtry`, the trees keep choosing the same strong components, so they are more alike. Node size matters much less. One observation changes the error by only 0.31%, so many of these differences are small, and the minimum over 48 settings is somewhat optimistic. PC2 and PC1 are the most important, with OOB error increases of about 0.127 and 0.101, followed by PC3, PC5, and PC6 at about 0.03.

Part c

```
test_prob <- predict(final_fit, Z_test)$ProbPrediction
test_pred <- factor(
  levels(y_train)[max.col(test_prob, ties.method = "first")],
  levels = levels(y_train)
)

print(table(Predicted = test_pred, Observed = y_test))
```

```
         Observed
Predicted   1   6   7
        1 102   0   0
        6   0 106   3
        7   2   0 104
```

```
print(mean(test_pred != y_test))
```

```
[1] 0.01577287
```

The test misclassification rate is 1.58% (5 of 317), close to the OOB error. Three 7s were predicted as 6 and two 1s as 7.

Question 3 Answers

Part a

I would keep the skill short and assume a capable AI already knows the standard steps of a random-forest analysis, such as bootstrapping, OOB error, and permutation importance. Long instructions for those steps add little and can override better choices the AI would make for a particular dataset. I would spend most of the skill on what it cannot guess: the required inputs, my tuning and selection rules, and how the report should look. A few basic instructions are still worth keeping because errors there are costly, like protecting the test set and setting a seed, and they would help a less capable AI. If the skill were for a more limited AI, I would add more of the basic steps, but I would keep my personal preferences the same.

Part b

1. Ask for the data file, response, predictors, seed, and output path, and never guess variable roles.
2. Check that the response type matches the forest model, and report n, p, class counts, and missing values.
3. Set aside a test set before fitting and use it once, only for the final model.
4. Use ordinary random forests with at least 1,000 trees, and report the package version and every non-default setting.
5. Tune `mtry` and node size by OOB error on a grid wide enough that the chosen values are not on its edge.
6. Report the regression test MSE, or the classification test error with a confusion matrix, next to a baseline model.
7. Among settings within a small margin of the lowest OOB error, choose the simplest one: larger node size, then smaller `mtry`.
8. Present OOB error for every tuning setting in one table, and mark the selected row.
9. Show only the top 10 predictors in a horizontal permutation importance plot with a colorblind-safe palette.
10. Write a one-page, 11-point PDF with hidden code, in the order setup, figure, table, results, limitations, and keep the interpretation to 3-4 plain sentences.